Experiment-2

What is Shell?

A **shell** is a command-line interpreter that acts as an interface between the user and the operating system. It accepts user commands, interprets them, and passes them to the kernel for execution. It also supports shell scripting to automate repetitive tasks.

Different types of Shells

- **Bourne Shell (sh):** The original Unix shell, widely used for scripting.
- **Bourne Again Shell (bash):** An enhanced version of sh; the default shell in most Linux distributions.
- **C Shell (csh):** Uses C-like syntax and supports command history and aliases.
- **Korn Shell (ksh):** Combines features of both Bourne and C shells with improved scripting capabilities.
- **Z Shell (zsh):** An advanced shell with features like auto-completion, spell correction, and customizable prompts.

Aim: To write a shell program to check if given number is even or odd.

Description: This experiment uses read for user input, the if-else conditional statement for decision-making, and arithmetic evaluation with (( )). The modulus (%) operator is used to check divisibility by 2, and echo is used to display whether the entered number is odd or even.

Algorithm:

1) Start.
2) Read a number from the user.
3) Check whether the number is divisible by 2.
4) If the remainder is 0, display "Even Number".
5) Otherwise, display "Odd Number".
6) Stop.

Program:

prime.sh × even

```
1 echo -n "Enter number: "
2 read n
3 if [ $((n % 2)) -eq 0 ];
4 then
5 echo "$n is  a even number"
6 else
7 echo "$n is odd number"
8 fi
```

Output:

```
cselab81@cselab8-OptiPlex-3060:~/Desktop/1601-24-733-327$ ./even.sh
Enter number: 10
10 is  a even number
cselab81@cselab8-OptiPlex-3060:~/Desktop/1601-24-733-327$
```

2. Aim: To write a shell program to check if given number is prime.

Description: This experiment uses shell scripting syntax such as read for input, the for loop to test divisibility, and the if-else statement for decision-making. Arithmetic evaluation with (( )) and the modulus (%) operator are used to determine whether the entered number is prime, while echo displays the result.

Algorithm:

1. Start.
2. Read a number n from the user.
3. If n is less than or equal to 1, display "Not a Prime Number" and stop.
4. Check whether n is divisible by any number from 2 to n/2.
5. If divisible, display "Not a Prime Number"; otherwise, display "Prime Number".
6. Stop.

Program:

prime.sh | even.sh

```
1 echo -n "Enter number: "
2 read n
3 is_prime=1
4 for ((i=2; i<n; i++))
5 do
6     if [ $((n % i)) -eq 0 ]; then
7         is_prime=0
8         break
9     fi
10 done
11 if [ $is_prime -eq 1 ]; then
12     echo "$n is a prime number"
13 else
14     echo "$n is not a prime number"
15 fi
```

Output:

```
cselab81@cselab8-OptiPlex-3060:~/Desktop/1601-24-733-327$ ./prime.sh
Enter number: 13
13 is a prime number
cselab81@cselab8-OptiPlex-3060:~/Desktop/1601-24-733-327$
```

3. Aim: To write a shell program to find factorial of given number.

Description: This experiment uses shell scripting syntax such as read for user input, a for loop to repeatedly multiply numbers, and arithmetic evaluation with (( )) to calculate the factorial. The echo command is used to display the computed factorial of the given number.

Algorithm:

1) Start.
2) Read a number n from the user.
3) If n is less than 0, display "Factorial is not defined for negative numbers" and stop.
4) Initialize fact = 1.
5) Multiply fact by each number from 1 to n.
6) Display the factorial of the given number.
7) Stop.

Program:

prime.sh × ever

```
1 echo -n "Enter number: "
2 read n
3 fact=1
4 for ((i=1; i<=n; i++))
5 do
6 fact=$(( fact * i ))
7 done
8 echo "$n! = $fact"
```

Output:

```
cselab81@cselab8-OptiPlex-3060:~/Desktop/1601-24-733-327$ ./fact.sh
Enter number: 5
5! = 120
cselab81@cselab8-OptiPlex-3060:~/Desktop/1601-24-733-327$
```

4. Aim: To write a shell program to check if given number is an Armstrong number.

Description: This experiment uses read for input, while loops to process each digit, and arithmetic evaluation with (( )). The modulus (%), division (/), and exponentiation (**) operators are used to calculate the sum of each digit raised to the power of the total number of digits, while if-else determines whether the number is an Armstrong number.

Algorithm:

1) Start.
2) Read a number from the user.
3) Count the number of digits in the number.
4) Find the sum of each digit raised to the power of the number of digits.
5) Compare the sum with the original number.
6) If both are equal, display "Armstrong Number"; otherwise, display "Not an Armstrong Number".
7) Stop.

Program:

prime.sh × even.sh

```
1 echo -n "Enter a number: "
2 read n
3 num_digits=${#n}
4 temp=$n
5 sum=0
6 while [ $temp -gt 0 ]
7 do
8     digit=$(( temp % 10 ))
9     sum=$(( sum + digit ** num_digits ))
10    temp=$(( temp / 10 ))
11 done
12 if [ $sum -eq $n ]; then
13    echo "$n is an Armstrong number"
14 else
15    echo "$n is NOT an Armstrong number"
16 fi
```

Output:

```
cselab81@cselab8-OptiPlex-3060:~/Desktop/1601-24-733-327$ ./armstrong.sh
Enter a number: 153
153 is an Armstrong number
cselab81@cselab8-OptiPlex-3060:~/Desktop/1601-24-733-327$
```

5. **Aim:** To write a shell program to check if given number is palindrome.

Description: This experiment uses read for input, while loops to reverse the digits of a number, and arithmetic evaluation with (( )). The modulus (%) and division (/) operators are used to extract and manipulate digits, while the if-else statement compares the reversed number with the original to determine whether it is a palindrome.

Algorithm:

1) Start.
2) Read a number from the user.
3) Store the original number in another variable.
4) Reverse the digits of the number.
5) Compare the reversed number with the original number.
6) If both are equal, display "Palindrome Number"; otherwise, display "Not a Palindrome Number".
7) Stop.

Program:

prime.sh × even.sh

```
1 echo -n "Enter a number: "
2 read n
3 temp=$n
4 rev=0
5 while [ $temp -gt 0 ]
6 do
7     rem=$(( temp % 10 ))
8     rev=$(( rev * 10 + rem ))
9     temp=$(( temp / 10 ))
10 done
11 if [ $n -eq $rev ]; then
12     echo "$n is a palindrome number"
13 else
14     echo "$n is not a palindrome number"
15 fi
```

Output:

```
cselab81@cselab8-OptiPlex-3060:~/Desktop/1601-24-733-327$ ./palindrome.sh
Enter a number: 151
151 is a palindrome number
cselab81@cselab8-OptiPlex-3060:~/Desktop/1601-24-733-327$
```

6. Aim: To write a shell program to print n terms of Fibonacci series.

Description: This experiment uses read to accept the number of terms and a while loop to generate the Fibonacci series. Arithmetic evaluation using $(( )) calculates the next term, while ((ct++)) increments the loop counter and echo displays the series.

Algorithm:

1) Start.
2) Read the number of terms n from the user.
3) Initialize the first two Fibonacci numbers as a = 0 and b = 1.
4) Repeat the loop for n terms.
5) Print the current term and calculate the next term using p = a + b.
6) Update a and b with the next Fibonacci values.
7) Stop.

Program:

prime.sh × even.sh

```
fib() {
    if (( $1 <= 1 )); then
        echo $1
        return
    fi
    a=$(fib $(($1 - 1)))
    b=$(fib $(($1 - 2)))
    echo $(( a + b ))
}
echo -n "Enter the number of terms: "
read n
echo "Fibonacci series up to $n terms:"
for (( i=0; i<n; i++ ))
do
    echo -n "$(fib $i) "
done
echo ""
```

Output:

```
cselab81@cselab8-OptiPlex-3060:~/Desktop/1601-24-733-327$ ./fibo.sh
Enter the number of terms: 5
Fibonacci series up to 5 terms:
0 1 1 2 3
cselab81@cselab8-OptiPlex-3060:~/Desktop/1601-24-733-327$
```